Untitled3.ipynb

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

[1] ✓ 0s

```python
import numpy as np

# THE PROBLEM: A simple mock irrigation rule to test our algorithm
# Goal: Find the best water amount (in mm) for 5 days.
# Let's say the perfect amount of water per day is 5.0 mm.
def calculate_error(schedule):
    perfect_schedule = np.array([5.0, 5.0, 5.0, 5.0, 5.0])
    # The error is how far away our schedule is from the perfect schedule
    return np.sum((schedule - perfect_schedule) ** 2)

# --- ALGORITHM START ---

# S1=INITIALISE CONFIGURATION
num_particles = 10
days = 5
max_iterations = 50

# Initialise arrays to hold positions and velocities
# Rows = particles, Columns = daily water amounts (0 to 10 mm)
positions = np.random.uniform(0, 10, (num_particles, days))
velocities = np.random.uniform(-1, 1, (num_particles, days))

# Initialise personal bests for each particle with their starting positions
p_best_positions = np.copy(positions)
p_best_scores = np.array([calculate_error(p) for p in positions])

# S2=CHOOSE THE BEST INITIAL GLOBAL VALUE
best_particle_index = np.argmin(p_best_scores)
g_best_position = np.copy(p_best_positions[best_particle_index])
g_best_score = p_best_scores[best_particle_index]

# STEP 3: THE MAIN ITERATION LOOP (UPDATE BEST VALUES)
for iteration in range(max_iterations):
    for i in range(num_particles):
```

Untitled3.ipynb

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

[1] ✓ 0s

```
            # Update Velocity
            r1, r2 = np.random.rand(days), np.random.rand(days)
            inertia = 0.7 * velocities[i]
            cognitive = 1.5 * r1 * (p_best_positions[i] - positions[i])
            social = 1.5 * r2 * (g_best_position - positions[i])

            velocities[i] = inertia + cognitive + social

            # B. Update Position
            positions[i] = positions[i] + velocities[i]
            positions[i] = np.clip(positions[i], 0, 10) # Keep water between 0mm and 10mm

            # C. Evaluate Current Score
            current_score = calculate_error(positions[i])

            # D. Update Personal Best Value if this new position is better
            if current_score < p_best_scores[i]:
                p_best_scores[i] = current_score
                p_best_positions[i] = np.copy(positions[i])

            # E. Update Global Best Value if this new position beats the whole swarm
            if current_score < g_best_score:
                g_best_score = current_score
                g_best_position = np.copy(positions[i])

    # --- RESULTS ---
    print("--- Final Optimized Irrigation Schedule ---")
    for day in range(days):
        print(f"Day {day+1}: Apply {g_best_position[day]:.2f} mm of water")

    print(f"\nFinal Error Score: {g_best_score:.6f} (Target is 0.00)")
```

```
--- Final Optimized Irrigation Schedule ---
Day 1: Apply 4.98 mm of water
Day 2: Apply 5.02 mm of water
Day 3: Apply 4.99 mm of water
Day 4: Apply 5.02 mm of water
Day 5: Apply 4.99 mm of water

Final Error Score: 0.001415 (Target is 0.00)
```

16/10/26

Agriculture Irrigation Scheduling

Algorithm

for each particle i in the swarm
    initialize pos[i] randomly
    initialize velocity[i] to zeros

    evaluate fitness (pos[i])

    personal_pos(i) = pos[i]
    personal_fitness[i] = fitness(pos[i])

global_best_pos = position of particle with best fitness in the entire.

for iteration = 1 to maxIterations:
    for each particle i in the swarm:
    //update particle's movement velocity
    //r1, r2 are random no:'s b/w 0 & 1
    // c1 (cognitive coefficient), c2 (social coefficient), w (inertia weight)
    velocity[i] = (w * velocity[i]) + (c1 * r1 * (personalbest_pos[i] - position[i])) + (c2 * r2 * (globalbest_pos - position[i]))
    pos[i] = pos[i] + velocity[i]

    //enforce irrigation boundaries
    clamp pos[i] within [min_water_required, max_system_capacity]
    current_fitness = fitness(position[i]) //new schedule

    if current_fitness is better than personalbest_fitness[i]:
        personal best_pos[i] = pos[i]
        personal best_fitness[i] = current_fitness

17/10/26

    if current_fitness is better than global_best_fitness:
        global best_pos = pos[i]

return global best_position.

Step1: select random tiger population

Step2: read the chromosome values for e

Step3: apply fitness values by fitness function. calculate the scores for each tiger.

Step4: see if there is tiger with same score
yes - mates them as optimal solution for offsprings

Step5: if no them choose more suitable one which contribute to fitness values

Step6: use the optimal tigers for crossover and produce new generation

The generation produce it fittest to environment then previous generation.

```
import random
genelength = 8
mutation_rate = 0.05
optimum = [1, 1, 0, 0, 1, 0, 0]
class Tiger:
    def __init__(self, dna):
        self.dna = dna
    def fitness(self):
        matches = sum(a == b for a, b in zip(self.dna, optimum))
        return (matches / genelength) * 100

def compatibility(male, female):
    differences = (sum(a != b for a, b in zip(male.dna, female.dna))
    return (differences / genelength) * 100

def crossover(male, female):
    point = random.randint(1, genelength - 1)
    child_dna = male.dna[:point] + female.dna[point:]
    return Tiger(child_dna)

def mutate(child):
    for i in range(gene_length):
```